第 8 章練習　主記憶體

本檔只含題目。請先自行建立狀態或變數表，再作答；解答在獨立檔案。

OS08-E01　Paging 轉換

4 KiB page size，virtual address 0x2A7C 的 page number 是 2，page table[2]=(frame=9, valid, read-only)。

1. 計算 offset 與 physical address。
2. 若程式執行寫入，會發生什麼？
3. 若 page table[2] 為 invalid，狀況是否一定是設計錯誤？

OS08-E02　有效存取時間

TLB lookup=2 ns，memory access=100 ns，兩者依序執行；TLB miss 需一次額外 page-table memory access。假設每次記憶體存取的成本相同、沒有其他 cache 或 page-walk cache，且前兩小題均不發生 page fault。令 TLB 命中率 h=0.95。

1. 建立 EMAT 公式並求值。
2. 若 h 降到 0.80，EMAT 為何？
3. 頁面替換成本出現後，模型如何修正？

OS08-E03　Page 大小取捨

一個工作有 100 個大小均為 33 KiB 的物件，每個物件獨立以 page 邊界配置、彼此不共用尾頁，page size 為 4 KiB；另比較 64 KiB page。

1. 以 4 KiB page 配置單一物件時最多浪費多少？
2. 為何 64 KiB page 可能減少 table 項目？
3. 計算 64 KiB page 每個物件與全部物件的尾端浪費。較小 page 是否一定較省記憶體？區分空間占用與轉換、缺頁所需時間。

OS08-E04　陌生 x86-64 位址的完整 walk

固定 x86-64、4-level、4 KiB pages、48-bit canonical VA；所有存取均為 CPL=3（user mode）。CPU 支援 NX，且 IA32_EFER.NXE=1；SMEP、SMAP、PKE 等額外機制均不干預。CR3=0x7000；下一層 bases 依序是 PDPT=0x8000、PD=0x9000、PT=0xA000。輸入 VA=0x00001234_5678_9ABC，PTE 的 frame=0x12AB，且四層均 present、user 可讀寫、NX=0。變更 PTE 後先完成所有可能快取該映射的 CPU 上必要的 TLB 失效與同步，再進行測試。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1. 檢查 canonical，列出 PML4/PDPT/PD/PT indices 與 offset。
2. 依 entry_size=8 計算四個 entry addresses；給出合理 entry values。
3. 推導 physical address。
4. 若 PML4 的 index 不變，但 PTE 改成 `0x12AB005`，read/write 結果如何？

OS08-E05　canonical、present 與權限的分流

沿用 OS08-E04 的模式、CPL=3、NXE=1 與權限假設，四個案例彼此獨立；更改表項後先完成受影響 CPU 的 TLB 失效與同步。測試：(A) VA 0x00002468_ACE0_1234；(B) E04 的 canonical 位址，但該路徑 PML4E.P=0；(C) E04 原映射且 PTE.NX=1；(D) VA=0x00008000_00000000。A 的表項內容未給出，不能假設它沿用 E04 的路徑。

1. A 的 indices/offset 為何？是否 canonical？
2. 哪個案例在 page walk 前就被拒？哪個在某一層 page fault？
3. 哪些資料 read/write 可完成，哪些 instruction fetch fault？
4. 如果只檢查 `present`，能否同時辨識 write protection 與 NX？補充 NXE=0 時 PTE bit 63 的處理，並以表格說明分支。

OS08-E06　ARM table layout、domain 與權限

固定 ARMv7-A AArch32 short descriptors、TTBCR.N=0、TTBCR.PD0=0、SCTLR.AFE=0、SCTLR.TRE=0、4 KiB small pages，兩次存取均為 PL0，DACR.D3=Client。P1 root=0x14000、L2=0x1C000，P2 root=0x18000、L2=0x1C400。兩筆 L1 raw words 為 0x1C061、0x1C461，兩筆 L2 raw word 都是 0x54862。P1 VA=0x004A6000，P2 VA=0x007C8000。兩程序使用互異且未重用的 ASID，對 frame 0x54000 恰有這兩筆映射；沒有其他映射、pin 或其他保持 frame 存活的引用。本題只追蹤 mapping count，初值為 2；它不等同一般實作的完整 refcount。解除映射後先完成所有受影響核心所需的 TLB invalidation、barrier 及同步，再測試或回收。

1. 先證明 root/L2 storage 的大小、對齊與區間不重疊，再計算 P1/P2 的 L1/L2 indices 與四個 entry addresses。
2. 逐欄解碼 L1/L2 raw words；small-page AP[2:0]必須用bit9與bits[5:4]，列出兩個read、write的結果。
3. 只移除 P1 的 L2 entry 並完成上述 TLB 維護後，P2 read、P1 read 與 mapping count 各如何？何時可回收 frame？
4. 若 DACR.D3 改成 Manager，或 SCTLR.AFE 改成1，原本的 write fault 結論是否仍成立？

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 8. Intel 64 and IA-32 SDM, Vol. 3A; Arm Architecture Reference Manual ARMv7-A/R.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study